RESEARCH ARTICLE | APRIL 29 2024

Restricted global optimization for QAOA

Peter Gleißner ; Georg Kruse ; Andreas Roßkopf

APL Quantum 1, 026106 (2024)
https://doi.org/10.1063/5.0189374

Restricted global optimization for QAOA

Cite as: APL Quantum 1, 026106 (2024); doi: 10.1063/5.0189374
Submitted: 28 November 2023 • Accepted: 4 April 2024 •
Published Online: 29 April 2024

Peter Gleißner,[a] Georg Kruse,[b] and Andreas Roßkopf[c]

AFFILIATIONS

Fraunhofer Institute for Integrated Systems and Device Technology IISB, Erlangen, Germany

[a]Electronic mail: peter.gleissner@tu-ilmenau.de
[b]Author to whom correspondence should be addressed: georg.kruse@iisb.fraunhofer.de
[c]Electronic mail: andreas.rosskopf@iisb.fraunhofer.de

ABSTRACT

The Quantum Approximate Optimization Algorithm (QAOA) has emerged as a promising variational quantum algorithm for addressing NP-hard combinatorial optimization problems. However, a significant limitation lies in optimizing its classical parameters, which is in itself an NP-hard problem. To circumvent this obstacle, initialization heuristics, enhanced problem encodings and beneficial problem scalings have been proposed. While such strategies further improve QAOA's performance, their remaining problem is the sole utilization of local optimizers. We show that local optimization methods are inherently inadequate within the complex cost landscape of QAOA. Instead, global optimization techniques greatly improve QAOA's performance across diverse problem instances. While global optimization generally requires high numbers of function evaluations, we demonstrate how restricted global optimizers still show better performance without requiring an exceeding amount of function evaluations.

I. INTRODUCTION

Quantum algorithms like Shor's algorithm[1] and the Harrow–Hassidim–Lloyd (HHL) algorithm[2] show exponential advantages over their classical counter parts. Since such quantum algorithms demand fault-tolerant quantum computers, variational quantum algorithms have emerged for currently available Noisy Intermediate-Scale Quantum (NISQ) devices, which combine parameterized quantum circuits and classical optimizers. One of these variational quantum algorithms is the Quantum Approximate Optimization Algorithm (QAOA), which is considered a promising candidate to show quantum advantage for NP-hard combinatorial optimization problems. While extrapolations suggest a quantum speed-up for problem instances that require several hundreds of qubits,[3] the main limitation lies in the exponential cost of optimizing the classical parameters of this quantum algorithm, which has been shown to be a NP-hard problem itself.[4] The cost landscape of QAOA exhibits wide plane areas and large numbers of local minima,[5] which are especially disadvantageous for local optimizers. To overcome this issue, heuristic strategies for the classical optimization as well as favorable problem formulations and encodings have been studied.[6–10] While such strategies enhance the performance of QAOA, further improvements are crucial in order to apply QAOA to real-world optimization problems like the Unit Commitment (UC) problem,[11] the Traveling Salesperson (TSP) problem[12] or the Factory Layout (FL) problem.[13]

In classical optimization, it is well known that local optimization algorithms, which are solely based on local information, are generally not adequate for solving NP-hard problems.[14] Instead, global optimization algorithms which combine multi-point global search and problem oriented local search have been well established for such problems.[15,16]

In this work we show why current approaches with local optimizers show poor performance, even for simple problem instances. While a current line of research aims to improve the overall structure of the cost landscape,[6,7] we argue that such improvements cannot resolve the drawbacks caused by the use of local optimizers. We therefore introduce global optimizers and show that they greatly improve the performance of QAOA on various problem instances and sizes, at the cost of a higher number of function evaluations.

Our work is structured as follows: In Sec. II we motivate the need for better optimization techniques and discuss current approaches. In Sec. III we briefly discuss QAOA and the problem formulations used in this work. In Sec. IV, we show how the cost landscape of general problem formulations undergoes a

transformation based on both the selected encoding parameters as well as the general parameter choices of QAOA. In Sec. V we motivate the use of global optimizers on the basis of a simple example and benchmark their performance on three use cases in Secs. VI and VII. Finally, we conclude our findings in Sec. VIII.

II. RELATED WORKS

To overcome the difficulty of optimizing the classical parameters of QAOA, various optimization strategies as well as favorable problem encodings and formulations have been proposed. One line of research focuses on improving the structure of QAOA's cost landscape in order to reduce its difficulty for local optimizers. Brandhofer *et al.* show how different factors like scaling parameters, encoding strategies and mixer Hamiltonians can influence the shape of the cost landscape and therefore influence optimization performance.[6] Albeit many enhancements have been proposed, this line of research does not consider the underlying problem of optimizing QAOA's classical parameters with local optimizers.

Another line of research proposes heuristics for parameter initialization in order to minimize the effort of classical optimization. Nakanishi *et al.* combine such a heuristic with a layer-wise training approach,[8] where each layer of QAOA is optimized after the other, leading to improved results at the cost of higher number of optimization steps. Zhou *et al.* also propose a heuristic for a layer-wise initialization,[7] while Sack and Serbyn propose adapting concepts from Quantum Annealing to the initialization of classical parameters.[9] A similar approach is the idea of parameter transfer, where the classical parameters of QAOA are first optimized for a simpler problem instance and the resulting parameters are then used for the initialization of harder problem instances.[10,17,18]

Instead of improving optimization results of QAOA by enhanced encodings or heuristics, various works have tried to replace the local optimizers by other optimization approaches such as reinforcement learning.[19] Closest to our claims are the works of Refs. 20 and 21. Acampora *et al.* demonstrate how the use of evolutionary strategies enhances the performance of QAOA on the Max-Cut problem, and Rad *et al.* show how the optimization of classical parameters of QAOA can be separated into two distinct phases: a first global phase where a bayesian estimate is used to initialize the parameters of a second local phase. Both works, which can be considered to use global optimization approaches, show enhanced performance over local optimizers.

Besides these approaches, many concepts like warm-starting[22] have been introduced, which improve the solution quality of QAOA without addressing the difficulty of optimizing the classical parameters with local optimizers. However such techniques have shown to enhance the overall properties of the cost landscape of some problem instances such as Max-Cut,[23,24] thereby potentially mitigating the effects of using local optimizers.

III. PRELIMINARIES

The cost landscape of hybrid quantum–classical optimization algorithms like QAOA has been subject to detailed analyses. The specific shape changes with the problem at hand and its encoding into a Hamiltonian. To start from a common basis, Sec. III A introduces the general structure of QAOA and Sec. III B the general form of the problem formulations used in this work.

A. QAOA

The aim of the optimization algorithm is the preparation of a state $|\psi(\beta,\gamma)\rangle$ that encodes the solution to the given problem, where β and γ are parameter vectors that are optimized by a classical optimizer. The output state is prepared from an equal superposition by an ansatz that consists of layers. Each of them depends on the value of β_i and γ_i, where i denotes the index of the layer. The layers are repeated p times, with p being small for implementations on NISQ devices. Each layer $\hat{H}_{layer}$ consist of two parts: The cost and the mixing Hamiltonians $\hat{H}_C$ and $\hat{H}_M$ respectively [ref. Eq. (1)].

$$\hat{H}_{layer} = e^{-i\beta_i \hat{H}_M} e^{-i\gamma_i \hat{H}_C}. \tag{1}$$

Here, β_i parameterizes the mixer term, while γ_i parameterizes the cost term.

The complete circuit can be written as

$$|\psi(\beta,\gamma)\rangle = e^{-i\beta_p \hat{H}_M} e^{-i\gamma_p \hat{H}_C} \dots e^{-i\beta_1 \hat{H}_M} e^{-i\gamma_1 \hat{H}_C} |+\rangle^{\otimes n}. \tag{2}$$

For all cases considered in this work, the mixer Hamiltonian is defined simply as

$$\hat{H}_M = \sum_i \hat{\sigma}_i^x. \tag{3}$$

There are several other options to chose the mixer Hamiltonian to improve performance,[6] but in this work we want to focus on the general drawbacks of local optimizers and therefore only consider this simplified case.

The cost Hamiltonian on the other hand encodes the given problem and can be constructed from a quadratic unconstrained binary optimization (QUBO) formulation via the Ising-Model. The cost Hamiltonian will be constructed as

$$\hat{H}_C = \sum_{j<i} J_{ij} \hat{\sigma}_i^z \hat{\sigma}_j^z + \sum_i h_i \hat{\sigma}_i^z, \tag{4}$$

with J_{ij} being the factors of the quadratic terms of the i-th and j-th element and h_i the factors of the linear terms of the Ising-Model. The so constructed circuit generates states depending on the parameters β and γ. The quality of a given state depends on the expectation value of the cost function denoted as $C(\beta,\gamma)$. This cost function is minimized by the classical optimizer.

B. Problem formulation

The problem formulations of interest consist of an objective function and a set of constrains for a binary (or integer) optimization problem. One possibility of encoding a problem formulation into a cost Hamiltonian for QAOA is via the previous reformulation of the problem as a QUBO formulation. A general guide on doing this can be found in Ref. 25. In this work the cost function of a given QUBO will have the following general form:

$$c(\{x_i\}) = s\left(H_{cost}(\{x_i\}) + \sum_{j=1}^{n_{pen}} P_j H_{pen,j}(\{x_i\})\right). \tag{5}$$

Here x_i refers to the binary variables of the optimization problem. H_{cost} and $H_{pen,j}$ are the functions encapsulating the cost and the n_{pen} penalties. The factor s, as described in Ref. 17, is a scaling factor that is chosen, such that it ensures numerical stability in the optimization process (ref. Sec. IV A). Finally, P_j scales the respective penalty term to change the influence it has on the cost landscape as shown in Refs, 6, 12, and 26 (ref. Sec. IV B). A detailed description of the problem formulations of the UC, TSP and FL problem can be found in Appendix A.

C. Performance metrics

The performance of an optimization algorithm depends on two main factors: The quality of the solution found and the calculation cost of this solution. The latter can be easily quantified in our setting as the number of cost function evaluations (which corresponds to the number of circuit executions). Therefore, the number of function evaluations will be the measure of how costly it is to obtain a given solution. The quality of the solution is measured by the expectation value of the cost function divided by the cost of the minimal solution, so the normalized cost $\frac{C(\beta,\gamma)}{C_{\min}}$.[18] This metric has the benefit, that it is not dependent on the absolute value of the cost function. Due to the dropping of the constant offset in the construction of the cost function the minimal cost will always be negative, so the optimal value is 1. Given these performance metrics, a given optimizer should maximize the normalized cost while minimizing the number of function evaluations.

IV. COST LANDSCAPE

The shape of the cost landscape for a given problem depends on a set of parameters that are influenced both by the specific problem at hand and the characteristics of the algorithm being used. These parameters greatly influence the performance of classical optimization methods and require a thorough examination of their impact on the underlying cost landscape. As we show in Sec. V, even for well designed QAOA cost landscapes, optimization (especially for local optimizers) remains difficult. To obtain such well designed QAOA cost landscapes, the effects of various parameters will be studied in the following.

Problem dependent parameters like the power demand L of the UC problem or the adjacency matrix D of the TSP and FL problem are directly tied to the problem. Other parameters, like the scaling factor s or pentalty factors P_j, have to be chosen such that stable and successful optimization is achieved. On the other hand, algorithmic dependent parameters are (partly) independent of the problem instance and influence the characteristics of the QAOA cost landscape as well. In this work only qubit number n and layer count p are investigated, dropping the additional degrees of freedom stemming from the choice of mixer Hamiltonian and enhanced encoding strategies.

The visualization of the cost landscapes for a given problem formulation for QAOA with one layer is straightforward: The parameters β and γ are used as x and y axis respectively and the expectation value of the cost function $C(\beta,\gamma)$ is used as z axis (ref. Fig. 1). In order to plot higher dimensional cost landscapes we follow the ideas of Ref. 27 where two random vectors (θ_1 and θ_2) serve as axes needed to visualize parts of the cost landscape.

A characteristic example of a well designed cost landscape of QAOA with a single layer for a four Unit UC problem instance is depicted in Fig. 1(a): Due to the simple construction of the mixer Hamiltonian as described in Ref. 9, a clear periodicity in the direction of β can be seen. This is not the case in the direction of γ, where (depending on the Hamiltonian) many different frequency components will overlay.[28] Generally, the detection of the combined period for arbitrary Hamiltonians is unlikely. Again this can be seen in Fig. 1(a). Since the periodicity of γ for the Hamiltonians considered in this work is unknown, strategies of using parameter initializations similar to an annealing schedule, as proposed in Ref. 9, cannot be applied.

The similarity of the position of local minima in the cost landscapes depicted in Fig. 1 leads to a path of investigation where one tries to apply the knowledge of one found minimum in one problem instance to other problem instances. This includes the concepts from Refs. 10, 17, and 29 where this strategy is successfully demonstrated. These concepts have the caveat, that they need prior knowledge of solutions found for similar problems, so the initial problem of finding a primary good solutions remains the same, motivating the use of global optimizers.

A. Effect of scaling factor s

The magnitude of s [ref. Eq. (5)] has a direct effect on the scaling of the coefficients in the cost Hamiltonian. As described by Ref. 17, this stretches or compresses the cost landscape with respect to γ, when s is smaller or greater than 1 respectively. The comparison between Figs. 1(a) and 1(c) illustrates this phenomenon. The change in s by a factor of 10 compresses the landscape spanning from 0 to 2π down to 0 to around 0.4. Additionally, the magnitude of $C(\beta,\gamma)$ also changes, as the calculation of the cost function is also dependent on s. The resulting states and their corresponding solutions are not changed, so the minima remain the same, solely shifted by this parameter.

As the effectiveness of standard optimization algorithms often deteriorates if the variables do not have the same order of magnitude, s should be chosen such that all variables fullfill this requirement. In Fig. 1(c) a further increase of s would result in a cost landscape, where small changes in γ would have an even greater effect, making convergence difficult. In this work, the rescaling heuristic suggested in Ref. 17 is used to enhance the cost landscape, where s is chosen such that the absolute mean weight of the coefficients is scaled to 1. This ensures that at least one minimum lies in the range $\gamma \in [0, 2\pi]$, but also results in a trade-off between excluding better minima from the search space on one side and a smaller, but limited parameter space on the other hand.

B. Effect of penalty factors P_j

The penalty factors affects the number and size of the local minima: With rising values of P_j, more local minima are placed in the same segment of the parameter space. This is due to the increased value a invalid solution adds to the expectation value of the solution, as states representing wrong solutions become less desirable. This effect is depicted in Figs. 1(a) and 1(b), where the increased P_j results in a rougher surface. When choosing the value of P_j there is again a trade-off: If P_j is too low, invalid solutions will become

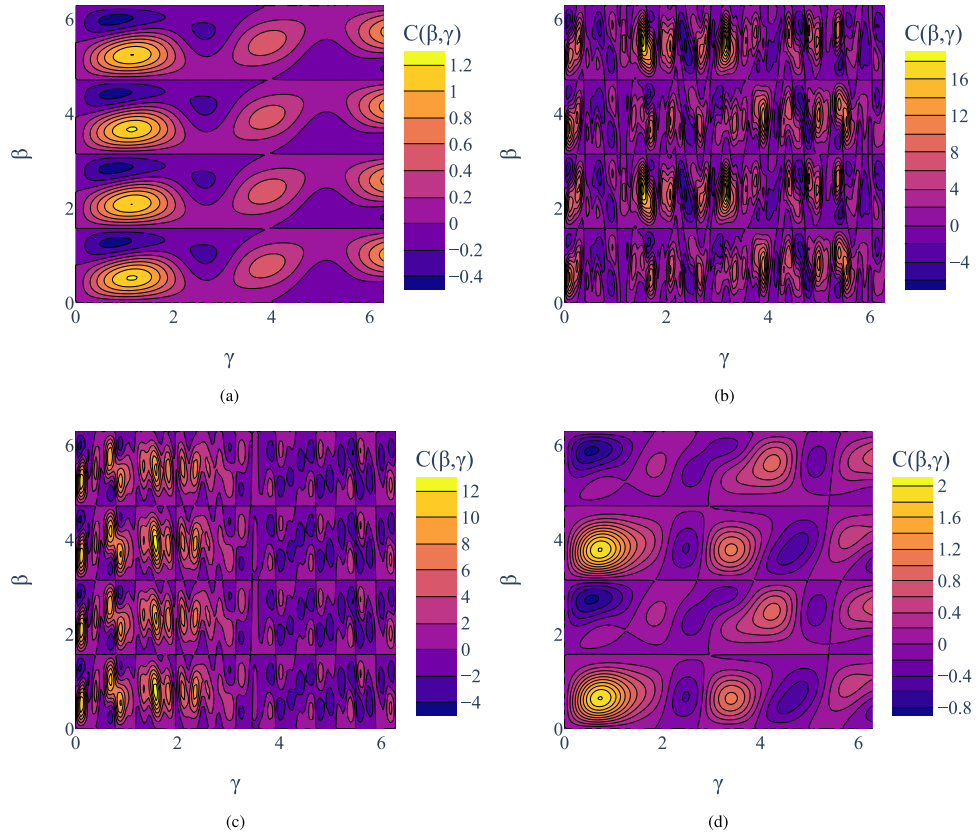

FIG. 1. Influence of problem dependent parameters: All contour plots depict cost landscapes created by a four Unit UC problem. Subfigure (a) serves as a *best case* baseline cost landscape with parameters $s = 0.0002$, $P = 0.2$ and $L = 300$ (chosen according to Sec. IV). Subfigures (b) and (c) show the introduction of additional minima with a changed values of $P = 2$ and $s = 0.002$ respectively. Subfigure (d) shows an additional cost landscape created by a related problem with $L = 200$. (a) Baseline $s = 0.0002$, $P = 0.2$, $L = 300$. (b) 10-times higher P. (c) 10-times higher s. (d) $L = 200$.

optimal (in terms of H_{cost}). If P_j is too large, the cost landscape becomes difficult to optimize in. Additionally, the relative difference between valid and optimal solutions decreases in comparison to the overall values of the cost landscape, so valid and optimal solutions become increasingly similar.

Works such as Refs. 6 and 26 have proposed methods to choose a favorable penalty factor for a given problem. Similarly Ref. 30 propose a method for choosing the penalty factors for various optimization problems, such that only valid solutions of the problem are optimal. In this work we follow the strategy suggested in Ref. 6, where P_j is iteratively increased by small values until the cheapest wrong (invalid) solution becomes at least as expensive as the optimal solution. A short synopsis to this method can be found in Appendix B 3. In real applications this strategy is not feasible as it requires knowledge of the full cost structure of the problem.

C. Effect of the problem dependent variables

In contrast to s and P_j, problem dependent variables such as the power demand L in the UC problem can not be chosen freely, as they are part of the problem at hand. The position of the minima of the cost landscape move depending on such values. To illustrate such changes, the power demand L of the UC problem has been varied between Figs. 1(a) and 1(d). While the underlying structure of the cost landscape remains the same, the distinct minima change. This also shows the difficulty of applying parameter transfer[10,17,29] for such problem instances, where some of the minima are greatly shifted or even disappear for varying problem dependent variables.

D. Effect of qubit number

For all tested problem instances, the increase of the number of qubits moves the archetype of the cost landscape from a turbulent shape with many local minima [Fig. 1(a)] to the shape of a barren plateau. Here, local minima occupy a smaller parameter subspace, while wide planes show low variance in the value of the cost function, causing the optimization in this domain to become difficult. This phenomenon can be seen in both Figs. 2(a) and 2(b), where the amount of minima drastically decreases at the cost of wide planes without any visible structure.

E. Effect of layer number

The effect of the number of layers is the hardest to access with the used method of visual analysis, as a higher number of layers

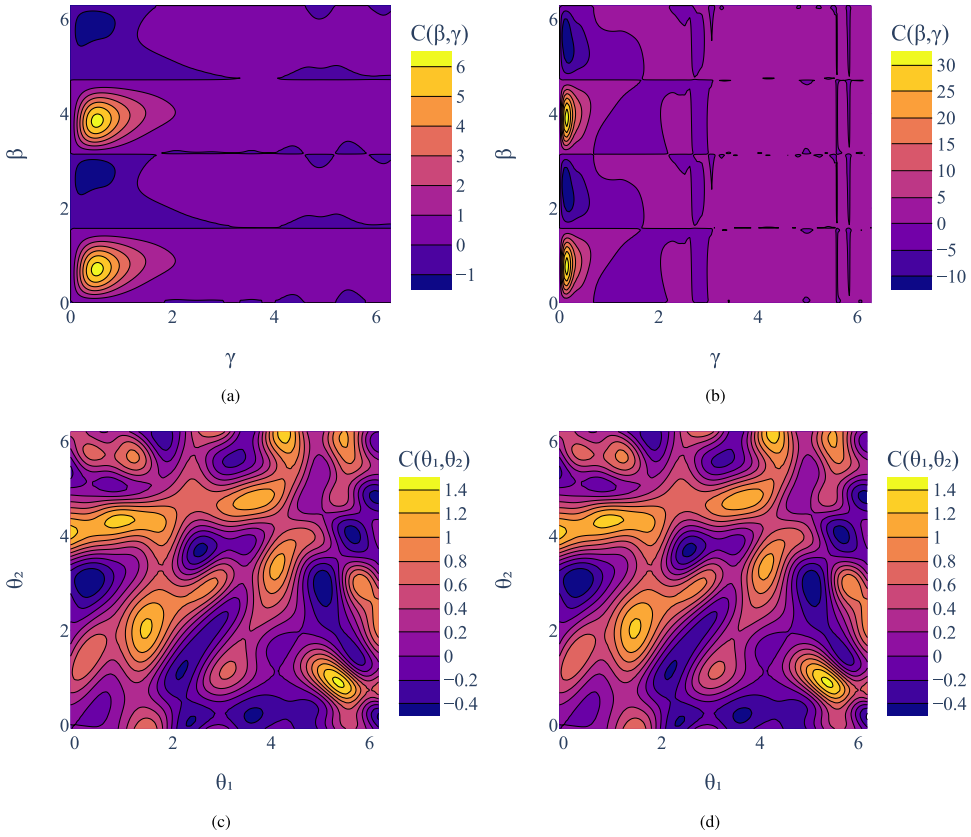

FIG. 2. Influence of algorithmic dependent parameters: All contour plots depict cost landscapes created by the UC problem at $s = 0.0002$, $P = 0.2$ and $L = 300$ [same as baseline in Fig. 1(a)]. Subfigures (a) and (b) show the effect more qubits have on the landscape. Subfigures (c) and (d) show the impact of an increase in layers, with the axis being random vectors in the parameter space. (a) 1 layer, 8 qubits. (b) 1 layer, 12 qubits. (c) 3 layers, 4 qubits. (d) 5 layers, 4 qubits.

results in higher dimensionality. Therefore, only limited information can be extracted from this comparison. The analysis shows, that the cost landscape exhibits an increased number of local minima, which is in line with the hypothesis from Ref. 31. An increase of number of free parameters makes the process of optimization increasingly complex. The two examples shown in Figs. 2(c) and 2(d) demonstrate this increased complexity.

V. GLOBAL OPTIMIZATION

Global optimization aims to find the best solution within the entire feasible search space. The search space encompasses all possible solutions of the problem and global optimization attempts to identify the global minimum by considering all variable combinations and constraints. However, global optimizers are often limited to finding the minimum in a designated area, leading to no guarantee of finding the global minimum.

On the other hand, local optimization focuses on finding the best solution within a specific region of the search space, usually centered around an initial guess or starting point. Un-like global optimization, local methods analyze the local behavior of the cost function near the starting point. While local optimization methods are generally faster and more efficient than global optimizers, as they do not explore the entire search space, they can get trapped in local minima. Additionally, their effectiveness is greatly dependent on their initialization.

Often there is an interplay between both types, as global optimizers can use local ones to improve the potential solutions found. There is no clear cut that defines all algorithms as part of one of the two groups. Methods-like the Univariate Marginal Distribution Algorithm (UMDA)[32] (considered as a local optimizer in this work), could be argued to also sample a part of the parameter space, which is a feature of global methods. A brief overview overall used local and global optimizers and their categorization can be found in Appendixes B 1 and B 2.

In the following we illustrate the drawbacks of local optimization for QAOA and motivate the use of global optimizers with a simple example. In Fig. 3 the cost landscape of an UC problem instance [previously depicted in Fig. 1(a)] can be seen. This cost landscape has been designed and optimized by the methods proposed by Refs. 6 and 17, as shown in Sec. IV, therefore representing a *best case*. A local optimization algorithm [represented here by the Nelder–Mead (NM) optimizer] is initialized at four different initial points and run until convergence. One can clearly see, that the local

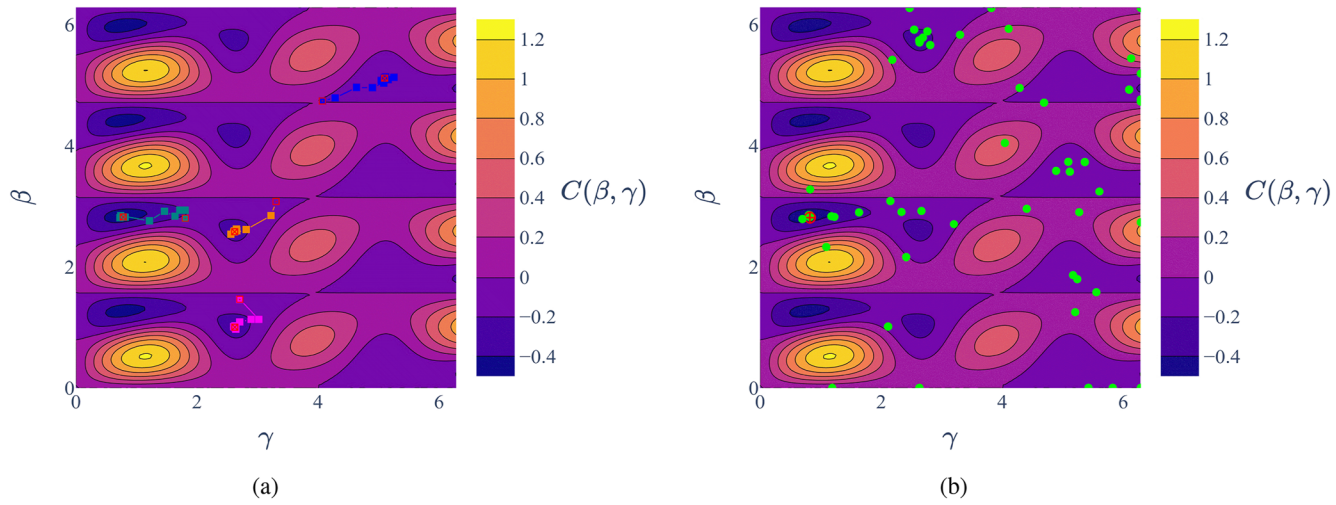

FIG. 3. Both surface plots show the cost landscape of the one layer, four unit UC problem baseline with $L = 300$, $s = 0.0002$ and $P = 0.2$ [as in Fig. 1(a)]. Subfigure (a) shows four optimization runs of the randomly initialized local optimizer NM, with the starting and final points indicated by a red marker. Depending on the initialization, the local optimizers only converge to the nearest local minimum. The second Subfigure (b) shows the sampling points (green) used in the bayesian optimization step of the FS optimizer and the selected final point for the local optimization step indicated by a red marker. The global optimizer finds one of the best possible minima. (a) Local optimization. (b) Global optimization.

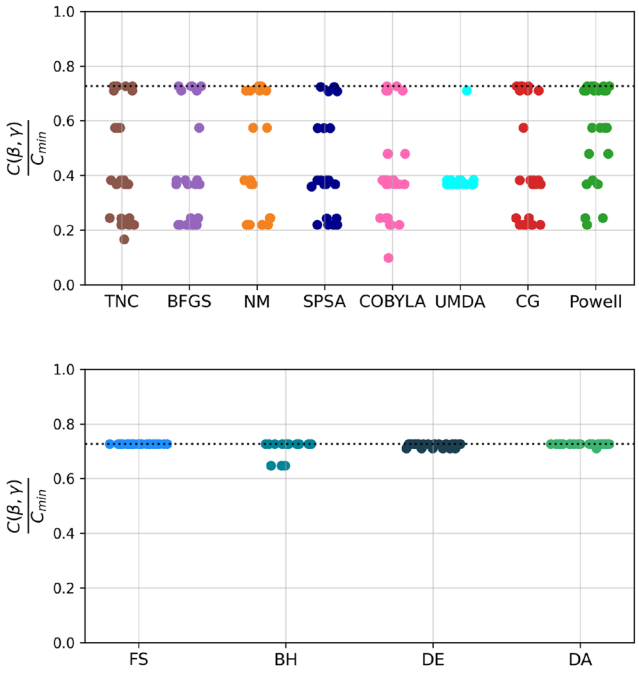

FIG. 4. Comparison of local (upper) and global (lower) optimizers on the baseline cost landscape (four unit UC problem, $L = 300$, $s = 0.0002$ and $P = 0.2$): The plots depict the results of 25 randomly initilizated runs of eight local and four global optimizers (see Appendix B). The dotted line denotes the value of the best possible minimum in the defined search space. The local optimizers get stuck in various local minima depending on their initializations, while global optimizers mostly find the optimal solution.

optimizer gets trapped in local minima, even for such simple problem instance. Their effectiveness, compared to global optimizers, is therefore greatly dependent on their initialization. In contrast to this, Fig. 3(b) shows the behavior of the global optimization algorithm Fast-Slow (FS).[21] This algorithm first samples globally and approximates the cost landscape by bayesian optimization. In a second step, a local optimizer is initialized at the most promising point. The yellow dots in Fig. 3(b) show the sampled points used for the global search, with the red dot denoting the most promising point for a local optimizer to start at. The global optimization phase ensures that the optimization algorithm is not trapped in local minima due to its (disadvantageous) initialization.

To illustrate this point further, different optimizers are used to find the minimum in the cost landscape of Fig. 4 with similar results. All local optimizers get trapped in various local minima, depending on their point of initialization. On the other hand, all global optimizers (almost always) find the global minima in the predefined parameter space. Note that in this scenario a value of 1 is not achievable, as the depth of QAOA as well as the choice of the parameter subspace does not allow such a solution. The improvement comes at the cost of an increasing number of function evaluations and time the optimization process takes. While local optimizers can achieve convergence within a few hundred function evaluations for simple problem instances, global optimizers might require several thousand evaluations to converge, if provided with the opportunity to do so. As this would take prohibitively long in simulations and especially in experiments on real hardware, this will be an important factor when comparing the different optimizers.

VI. EXPERIMENTS

In order to demonstrate the advantage of global optimization, experiments on three industrially motivated use cases are

performed. Most experiments were conducted on different configurations of the UC problem, which has the benefit of being scaleable to arbitrary numbers of qubits.

A. Use cases

For all use cases, the cost landscapes of the problem instances are enhanced according to the principles of Sec. IV and kept constant for all experiments. Differences between samples are solely due to the optimizer used and their initialization points. The generalization capability of global optimizers is shown not only on varying problem instances, but across different use cases all together.

B. Optimizers

Better solutions often come at the expense of higher number of function evaluations. These in turn influence the run time of an algorithm in simulations and on real hardware. Limiting the number of global optimization steps is therefore crucial. To balance this trade-off the global optimizers are partly set up such that results roughly require 1000–5000 function evaluations. Local optimizers are run for 200 optimization iterations each, with different numbers of function evaluations needed by each algorithm for one iteration. This restriction allows them to perform reasonably well. All optimizers described in Appendix B are evaluated in simulations on the UC problem use case, and the best global optimizers are selected. These optimizers are evaluated on the TSP and FL use case and further investigated on quantum hardware.

C. Backend

The simulations are carried out with an ideal statevector backend and a shot based noisy simulation backend. In the latter a custom noise model is used, which imitates the gate and readout errors described in qiskits *FakeBoeblingenV2* backend, without copying its coupling map, therefore allowing for all-to-all connections in the circuit. This model results in lower error rates than the real counterpart, but allows for easier scaleable and faster experiments. To verify the findings from simulations, experiments are also run on real hardware. All experiments use layer numbers of $p = 1$ and $p = 3$.

VII. RESULTS

A. Simulations

All optimizers are tested on 8, 10 and 14 qubit UC problem instances. Each optimizer is tested on 25 random problem instances and with random initial parameters. The results for state vector simulations are depicted in Fig. 6 in the Appendix. The global optimizers are evaluated with and without a limited number of function evaluations. In the following, just the abbreviations of the algorithms will be used (see Appendix B for details). On all problem instances the *unrestricted* global optimizers outperform the local optimizers, while requiring one to three orders of magnitude more function evaluations. The performance of the global optimizers Simplical Homology Global Optimization (SHGO) and Basin-Hopping (BH) is greatly decreased, as the number of allowed

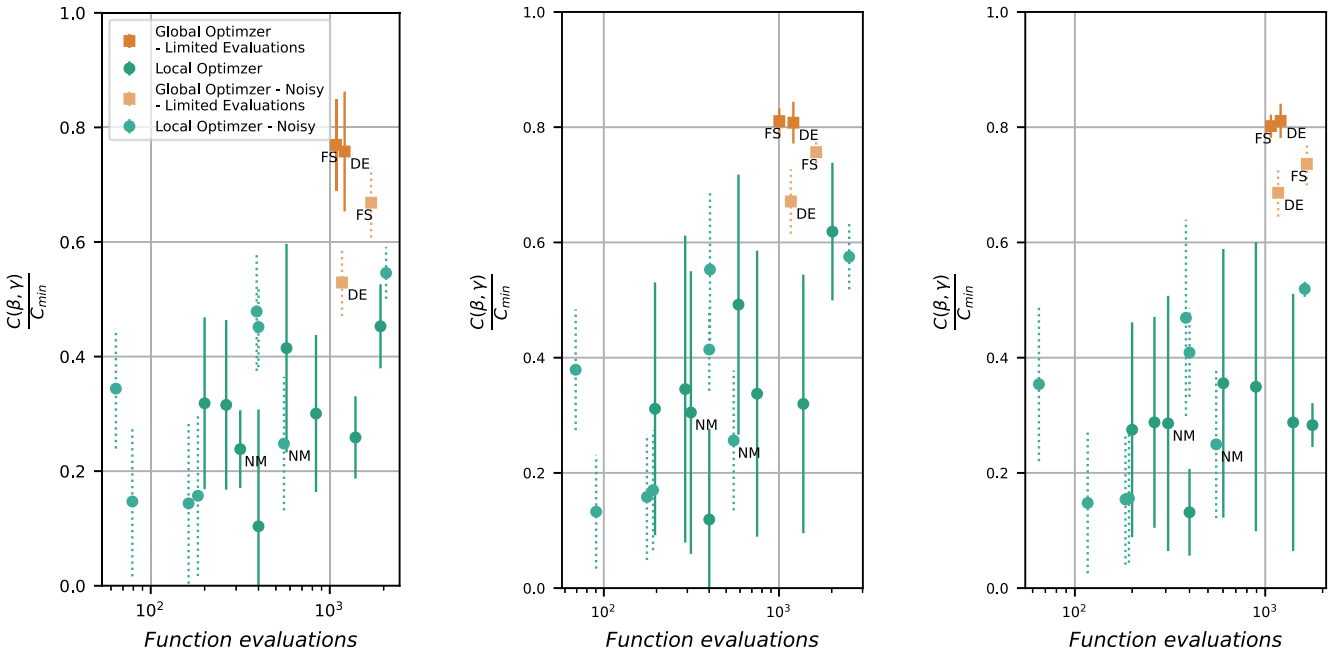

FIG. 5. Results of nine qubit use case instances of a three layer QAOA with adaptively chosen *s* and, if needed, an iteratively chosen penalty factor. State vector and noisy simulations for all tested local optimizers and the most promising global optimizers with ten random initializations each. NM is labeled as a reference to results in Table I. (a) UC. (b) TSP (c) FL.

TABLE I. Results of hardware experiments on *IBMQ Ehningen* (backend = e) and comparison with state-vector (backend = s) and noisy (backend = n) simulation on various UC problem instances with 10–18 qubits (Q). Two local optimizers (NM and Powell) are benchmarked against the *restricted* global optimizers FS and DE in terms of function evaluations (eval.) and normalized cost (norm. cost).

			NM		Powell		DE		FS	
Q	p	Back	Eval	Norm. cost	Eval	Norm. cost	Eval	Norm. cost	Eval	Norm. cost
10	1	s	80 ± 23	0.1 ± 0.23	101 ± 25	0.384 ± 0.33	613 ± 84	0.677 ± 0.048	612 ± 18	0.73 ± 0.043
10	1	n	525 ± 8	0.261 ± 0.116	105 ± 54	0.421 ± 0.145	1386 ± 445	0.693 ± 0.029	1489 ± 224	0.715 ± 0.017
10	1	e	533	0.293	⋯	⋯	414	0.352	971	0.411
10	3	s	317 ± 10	0.296 ± 0.195	590 ± 145	0.367 ± 0.217	1254 ± 85	0.767 ± 0.048	1179 ± 98	0.723 ± 0.068
10	3	n	558 ± 31	0.258 ± 0.104	371 ± 146	0.446 ± 0.096	1201 ± 27	0.493 ± 0.032	1657 ± 170	0.53 ± 0.061
10	3	e	546	0.046	⋯	⋯	1137	0.11	974	0.092
14	3	s	322 ± 14	0.277 ± 0.329	830 ± 238	0.367 ± 0.311	1230 ± 32	0.877 ± 0.002	1203 ± 34	0.898 ± 0.025
14	3	e	549	0.091	⋯	⋯	1200	0.216	992	0.172
18	3	s	318 ± 3	0.227 ± 0.351	636 ± 98	0.309 ± 0.302	1192 ± 147	0.922 ± 0.022	830 ± 104	0.905 ± 0.001
18	3	e	582	0.132	⋯	⋯	1270	0.142	990	0.142

function evaluations is reduced to 1000–5000. This is also the case for DE and DA, albeit this trend is attenuated especially for DE. The decrease in function evaluations leads especially for DA to a higher variance in performance across runs. Therefore the global optimizers FS and DE show the most promising behavior: high performance with low variance and relatively low number of function evaluations.

The global optimizers FS and DE are benchmarked in a *restricted* setting against all local optimizers on all three use cases (UC, TSP, FL) with state vector as well as noisy simulations. In Fig. 5 the comparison of the performance on nine qubit instances of each use case are depicted. All depicted optimizers are again tested on randomly generated problem instances with 25 random initial QAOA parameters each. Here a clear trend can be observed: The *restricted* global optimizers improve the performance of QAOA by a factor of two while increasing the number of function evaluations by roughly the same factor. At the same time the variance of performance is strongly reduced.

To mitigate the dependence of the solution quality of local optimizers on their random initialization, a common strategy is to evaluate several random initializations (in parallel) and then to select the best solution among these initializations. The caveat of this approach is the increase of function evaluations by the number of random initializations. Following this approach, we evaluate the optimizers Powell and Nelder–Mead with ten random initializations on the same random problem instance against one single random initialization of the two best performing global optimizers FS and DE. We test the performance across ten random initializations for each optimizer on the same problem instance. In Fig. 7 in Appendix B it can be seen, that the *restricted* global optimizers outperform the local optimizers in this scenario not only in terms of solution quality, but also in terms of function evaluations.

B. Hardware experiments

To verify the results of our simulations, we evaluated FS and DE as well as the local optimizer NM on the 27 qubits *IBMQ Ehningen* quantum computer. The results in Table I show that for all tested problem instances the global optimizers FS and DE outperform all tested local optimizers, while generally requiring more function evaluations. Among the different configurations, a deteriorating effect on the solution quality can be observed that is caused by hardware errors: The errors of deeper circuits even prevail the benefits of higher layer numbers, such that QAOA with $p = 1$ has higher values of $\frac{C(\beta,\gamma)}{C_{\min}}$ than its three layer counterpart. These findings are in line with the results in Ref. 18, where - depending on the problem - better performance with increasing layer number is not to be expected. The limited number of samples available for higher numbers of qubits makes it hard to estimate the scalability of this approach, so it remains to be shown, that the advantage holds for larger systems, even though the trend seems promising.

VIII. CONCLUSION

In this work, we investigated the performance of local optimizers on well designed cost landscapes of QAOA. We demonstrated that even for simple problem instances, local optimizers fail to find optimal solutions. Global optimizers on the other hand outperform local optimizers not only on such simple problem instances, but on a variety of use cases at the cost of higher numbers of function evaluations. These results hold for state vector simulations, noisy simulations as well as for experiments on real hardware. In order to overcome the caveat of high numbers of function evaluations required, we propose to restrict the global optimizers in terms of function evaluations. While these restrictions lead to a great decrease in number of function evaluations, the overall solution quality is only slightly decreased. Global optimizers like FS and

DE can greatly improve QAOA's performance, while increasing the number of function evaluations only by a factor of two compared to local optimizers like Powell or NM.

In current research, local optimization has been the standard approach to optimize the classical parameters of QAOA. Our work showed that restricted global optimizers are better suited and should be applied more widely. Especially combined with further enhancements in problem encodings, problem scalings and mixer designs, global optimizers can help to pave the way for the application of QAOA to real world problems.

ACKNOWLEDGMENTS

This project is supported by the Federal Ministry for Economic Affairs and Climate Action on the basis of a decision by the German Bundestag through the project Quantum-enabling Services and Tools for Industrial Applications (QuaST).

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Peter Gleißner: Data curation (equal); Investigation (equal); Methodology (equal); Resources (equal); Software (equal); Validation (equal); Visualization (equal); Writing – original draft (equal). **Georg Kruse**: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Project administration (equal); Resources (equal); Software (equal); Supervision (equal); Validation (equal); Visualization (equal); Writing – original draft (equal); Writing – review & editing (equal). **Andreas Roßkopf**: Conceptualization (equal); Funding acquisition (equal); Project administration (equal); Supervision (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

APPENDIX A: PROBLEM FORMULATIONS

1. Unit commitment problem

The *Unit Commitment* (UC) problem describes the allocation of power units to satisfy a given power demand L for the lowest possible price. The formulation used here is modeled after:[11]

$$H_{cost,UC}(\{x_i\}) = \sum_{i=1}^{n_{units}} (A_i + B_i p_i + C_i p_i^2) x_i. \tag{A1}$$

A_i, B_i and C_i are fixed parameters, that encode the cost for producing an amount of power p_i, if the unit x_i is turned on. In this work, p_i is additionally fixed to constant integer values, which are not subject to the optimization process. n_{units} denotes the overall number of units of the problem and is equal to the number of required qubits.

To Eq. (A1) constraines are added, to ensure that the sum of the produced power is equal to the power demand L, which can be expressed as the penalty term

$$H_{pen,UC,1}(\{x_i\}) = \left(\sum_{i=1}^{n_{units}} p_i x_i - L \right)^2. \tag{A2}$$

Equations (A1) and (A2) are used to construct the QUBO formulation using Eq. (5).

2. Traveling salesperson problem

The objective of the *Traveling Salesperson* (TSP) problem is to find the shortest possible route for visiting a list of cities and returning to the initial point. The used implementation is taken from Ref. 12 and is described there in greater detail. The binary variables $x_{i,j}$ of the problem have two indicies, with i being an identifier for the city and j the point in time, when it is visited. Both inidices i and j range from 1 to n_{cit}, with n_{cit}^2 qubits needed to construct the QAOA circuit. With this the problem can be described as

$$H_{cost,TSP}(\{x_{ij}\}) = \sum_{i,i',j=1}^{n_{cit}} D_{ii'} x_{ij} x_{i'(j+1)}. \tag{A3}$$

D is the adjacency matrix describing the distances between each city, with the element $D_{ii'}$ being the distance between the i^{th} and i'^{th} city. So each movement between cities in consecutive time steps adds to the cost. The constraints are added with the penalty term

$$H_{pen,TSP,1}(\{x_{ij}\}) = \sum_{i=1}^{n_{cit}} \left(1 - \sum_{j=1}^{n_{cit}} x_{ij} \right)^2 + \sum_{j=1}^{n_{cit}} \left(1 - \sum_{i=1}^{n_{cit}} x_{ij} \right)^2. \tag{A4}$$

Equation (A4) ensures that each city is only visited once and only one city is visited each time step respectively. The results in Ref. 12 show, that using the same penalty factor for both constraints works reasonably well, so this simplification will be used here as well. Combining Eqs. (A3) and (A4) in Eq. (5) constructs the QUBO formulation.

3. Factory layout problem

The *Factory Layout* (FL) problem is based on Ref. 13. A number of positions n_{pos} on a factory floor plan are given and some number n_{mach} of production units has to be placed as efficiently as possible to minimize the cost of transport between each machine. As in the previous case the distances between the positions can be summarized in an adjacency matrix D. The flow of material between each of the machines can be described by a transportation density matrix T, where each element $T_{ii'}$ describes the amount of material going from the i^{th} and i'^{th} production cell. To construct the problem, the decision variables have two indices with $x_{i,j}$ being the machine i placed on position j. Using these definitions the problem can be formulated as

$$H_{cost,FL}(\{x_{ij}\}) = \sum_{i,i'=1}^{n_{mach}} \sum_{j,j'=1}^{n_{pos}} D_{jj'} T_{ii'} x_{ij} x_{i'j'}. \tag{A5}$$

The constrains in this problem are the limitation of each machine being used exactly once and each position being at most taken once. This results in two distinct penalty terms: Each machine being used is written as

$$H_{pen,FL,1}(\{x_{ij}\}) = \sum_{i=1}^{n_{mach}} \left(\sum_{j=1}^{n_{pos}} x_{ij} - 1 \right)^2, \tag{A6}$$

while each position being taken once is encoded as

$$H_{pen,FL,2}(\{x_{ij}\}) = \sum_{j=1}^{n_{pos}} \sum_{i,i'=1}^{n_{mach}} x_{ij} x_{i'j}. \tag{A7}$$

APPENDIX B: ALGORITHMS AND HEURISTICS

1. Local optimizers

The used optimizers are summarized in short and if not noted otherwise, the standard Scipy implementation is used.

- **Nelder–Mead (NM)**:[33] This algorithm is derivative-free and uses a simplex of $n + 1$ points, with n being the dimensionality of the problem. Based on the function values of the different vertices a new point is constructed, that should be closer to the next minimum. The used Scipy implementation uses the version described in Ref. 34.
- **Powell**:[35] This algorithm is derivative-free and performs several line searches to find the next optimum. It is constructed as an iterative process of choosing lines to search, based on the currently known minimum and performing the actual searches. The Scipy implementation is a modified version, that has no closer description.
- **Conjugate Gradient (CG)**:[36] This algorithm uses derivatives and is originally designed to find solutions for big linear systems of equations. Based on gradient information it chooses a subspace to optimize for the next step.
- **Broyden–Fletcher–Goldfarb–Shannon (BFGS)**:[37] This algorithm approximates the gradients iteratively. Based on those the direction of descent is chosen and a line search is performed.
- **Truncated Newton (TNC)**:[38] This algorithm uses a CG-Method to solve the Newton equations and update the parameters for the next iteration based on this. It is designed to handle non linear functions as well.
- **Contrained Optimization BY Linear Approximation (COBYLA)**:[39] This algorithm is derivative-free and uses a linear approximation of the problem to find potential points. For this a simplex is constructed that forms the basis of the approximation. In each iteration, vertices can then be replaced based on the approximation to improve the simplex or to save a found vector.
- **Simultaneous Perturbation Stochastic Approximation (SPSA)**:[40] This algorithm approximates the gradient to follow by using only two measurements, so it is independent of dimensionality. With this gradient the next step is calculated and the approximation is performed again. The implementation used can be found in the *qiskit* package.
- **Continuous Univariate Marginal Distribution Algorithm (UMDA)**:[32] This algorithm populates an area with sampling points and uses the best as data in the approximation of the cost function with univariate normal distributions. From these combined distributions new points are drawn for the sampling of a new generation. It is part of the *qiskit* package.

2. Global optimizers

As with the local optimizers this work mostly utilizes optimizers from the Scipy library, with deviations noted in the following.

- **Basin-Hopping (BH)**:[41] This algorithm is initialized at some random point and tries to find the next local minimum with a local minimizer. After convergence the algorithm applies a random perturbation to the coordinates in order to find a different local minimum. Depending on the found function value the algorithm can chose to use this point as the basis for the next global step or reject the point and revert to a previous solution. The sequence of local optimizations and global steps is repeated until either the maximum number of steps is reached or some condition is met. The used implementation ignores if the local optimizer converged to a minimum or not, and decides on accepting a new point purely based on its function value.
- **Differential Evolution (DE)**:[42] This algorithm uses a random population of points in the search space to find the minimum. From iteration to iteration it combines the vectors of members of the current population to form members of the next generation.
- **Dual Annealing (DA)**:[43] This algorithm is also known as *Generalized Simulated Annealing*. It tests new random points, minimizes them with a local optimizer and rejects ones with higher function values than the current best point based on a random distribution. This distribution is annealed over time, so the condition of acceptance becomes stricter.
- **Simplical Homology Global Optimization (SHGO)**:[44] This algorithm uses sampling points to build a complex for approximating the cost function and finding potential regions where the minima might lie. It optimizes all possible points with a local optimizer to find all local minima and returns them.
- **Fast-Slow (FS)**:[21] This algorithm is a hybrid between a bayesian learning regime and a classical local optimizer. In a first step, the energy landscape is approximated over a wide area via bayesian approximation. With this model the most promising point is identified through classical optimization. This point will be minimized on the actual function by a local optimizer in a second step. This approach is the only one that is not directly implemented in the Scipy library. Instead, the bayesian learning from the *Scikit-learn* library is used for the first step, while the second step is done with the scipy optimizers.

3. Adaptive choice of penalty factors P_j

The goal of the algorithm mentioned in Ref. 6 is setting P_j just large enough, so the optimal solution becomes the global optimum

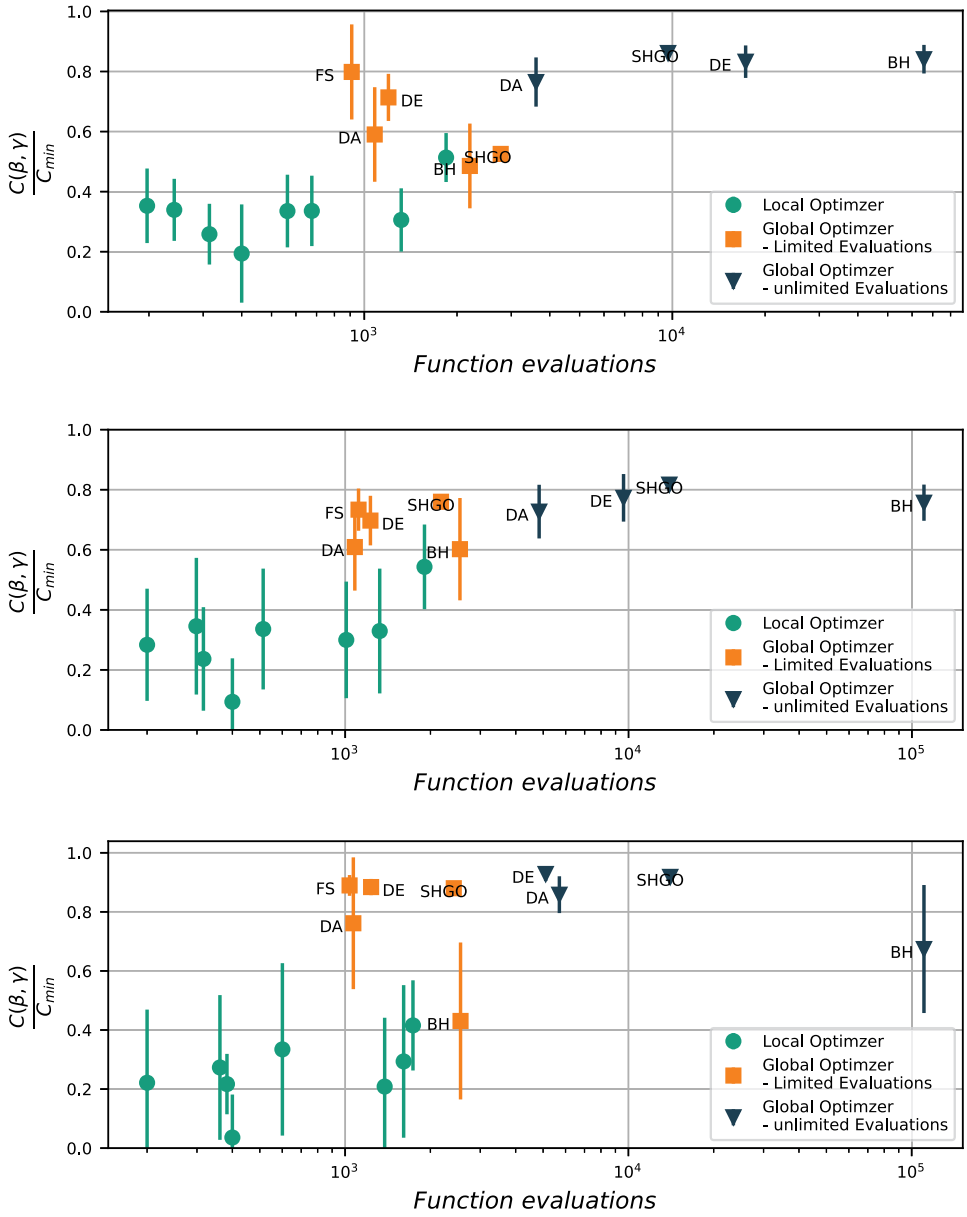

FIG. 6. Results on three instances of the UC use case with 8 (upper), 10 (middle) and 14 (lower) qubits with a three layer QAOA. All optimizers are tested with 25 random initializations. Global optimizers are either run with restrictions of 1000–5000 function evaluations or unrestricted.

of the function of Eq. (5). This does not necessarily mean, that all wrong (invalid) solutions have a higher value of the cost function than any of the valid solutions.

For the construction of P_j the cost structure of all combinations needs to be known beforehand. Out of this one can extract $H_{\min,opt}$, the cost of the optimal solution and $\overline{H}_{val}$, the mean cost of the valid solutions. With this the algorithm can start at a $P_j = 0$:

1. Determine the lowest cost of the wrong solutions $H_{\min,wrong}$
2. Check if following equation is true:

$$H_{\min,wrong} \geq \frac{1}{2}(H_{\min,opt} + \overline{H}_{val}) \tag{B1}$$

If yes, the algorithm terminates.

3. If the equation of the previous steps is not true, add to P the following:

$$\Delta P = \frac{\frac{1}{2}(H_{\min,opt} + \overline{H}_{val}) - H_{\min,wrong}}{\left(\sum_{i=1}^{N} p_i y_i^* - L\right)^2}. \tag{B2}$$

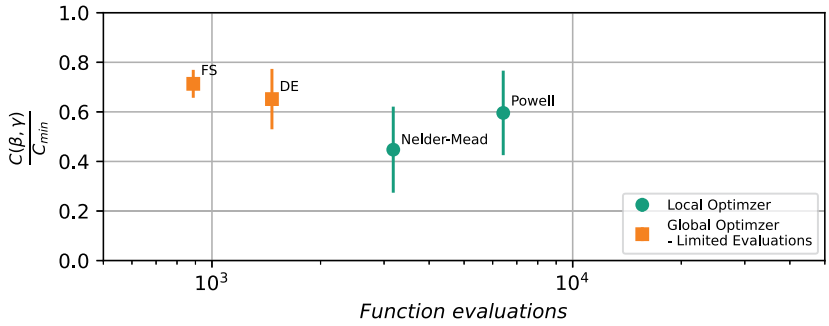

FIG. 7. Results on a randomly drawn nine qubit UC problem instance with a three layer QAOA. For each run of the optimizers Powell and Nelder–Mead, ten random initializations on the same random problem instance are optimized, selecting the best score across all initializiatons at the end. The global optimizers FS and DE are only run once for a single run. We plot the mean and variance of the performance across ten random initializations for each optimizer run on the same problem instance.

with y_i^* being the states of the wrong solutions with the lowest cost.

This approach is called *adaptive*, as it is adapting to the overall cost structure. It has the obvious drawback, that for this algorithm the cost structure has to be known beforehand and therefore can not be used in a real application.

REFERENCES

[1]P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM J. Comput. **26**, 1484–1509 (1997).
[2]A. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for solving linear systems of equations," Phys. Rev. Lett. **15**, 150502 (2008).
[3]G. Guerreschi and A. Matsuura, "QAOA for Max-Cut requires hundreds of qubits for quantum speed-up," Sci. Rep. **9**, 6903 (2019).
[4]L. Bittel and M. Kliesch, "Training variational quantum algorithms is np-hard," Phys. Rev. Lett. **127**, 120502 (2021).
[5]X. Ge, R.-B. Wu, and H. Rabitz, "The optimization landscape of hybrid quantum–classical algorithms: From quantum control to NISQ applications," Annu. Rev. Control **54**, 314 (2022).
[6]S. Brandhofer, D. Braun, V. Dehn, G. Hellstern, M. Hüls, Y. Ji, I. Polian, A. S. Bhatia, and T. Wellens, "Benchmarking the performance of portfolio optimization with QAOA," Quantum Inf. Process. **22**, 25 (2022).
[7]L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," Phys. Rev. X **10**, 021067 (2020).
[8]K. M. Nakanishi, K. Fujii, and S. Todo, "Sequential minimal optimization for quantum-classical hybrid algorithms," Phys. Rev. Res. **2**, 043158 (2020).
[9]S. H. Sack and M. Serbyn, "Quantum annealing initialization of the quantum approximate optimization algorithm," Quantum **5**, 491 (2021).
[10]M. Streif and M. Leib, "Training the quantum approximate optimization algorithm without access to a quantum processing unit," Quantum Sci. Technol. **5**, 034008 (2020).
[11]S. Koretsky, P. Gokhale, J. M. Baker, J. Viszlai, H. Zheng, N. Gurung, R. Burg, E. A. Paaso, A. Khodaei, R. Eskandarpour, and F. T. Chong, "Adapting quantum approximation optimization algorithm (QAOA) for unit commitment," in *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2021), pp. 181–187.
[12]L. Palackal, B. Poggel, M. Wulff, H. Ehm, J. M. Lorenz, and C. B. Mendl, "Quantum-assisted solution paths for the capacitated vehicle routing problem," arXiv:2304.09629 (2023).
[13]M. Klar, P. Schworm, X. Wu, M. Glatt, and J. C. Aurich, "Quantum annealing based factory layout planning," Manuf. Lett. **32**, 59–62 (2022).
[14]*State of the Art in Global Optimization: Computational Methods and Applications*, edited by C. Foudas and P. Pardalos (Kluwer Academic Publishers, Dordrecht, 2013).
[15]M. N. Omidvar, X. Li, and X. Yao, "A review of population-based metaheuristics for large-scale black-box global optimization—Part I," IEEE Trans. Evol. Comput. **26**, 802–822 (2022).
[16]B. Hartke, "Global optimization," Wiley Interdiscip. Rev.: Comput. Mol. Sci. **1**, 879–887 (2011).
[17]R. Shaydulin, P. C. Lotshaw, J. Larson, J. Ostrowski, and T. S. Humble, "Parameter transfer for quantum approximate optimization of weighted MaxCut," ACM Trans. Quantum Comput. **4**, 1 (2023).
[18]M. P. Harrigan, K. J. Sung, M. Neeley, K. J. Satzinger, F. Arute, K. Arya, J. Atalaya, J. C. Bardin, R. Barends, S. Boixo, M. Broughton, B. B. Buckley, D. A. Buell, B. Burkett, N. Bushnell, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, S. Demura, A. Dunsworth, D. Eppens, A. Fowler, B. Foxen, C. Gidney, M. Giustina, R. Graff, S. Habegger, A. Ho, S. Hong, T. Huang, L. B. Ioffe, S. V. Isakov, E. Jeffrey, Z. Jiang, C. Jones, D. Kafri, K. Kechedzhi, J. Kelly, S. Kim, P. V. Klimov, A. N. Korotkov, F. Kostritsa, D. Landhuis, P. Laptev, M. Lindmark, M. Leib, O. Martin, J. M. Martinis, J. R. McClean, M. McEwen, A. Megrant, X. Mi, M. Mohseni, W. Mruczkiewicz, J. Mutus, O. Naaman, C. Neill, F. Neukart, M. Y. Niu, T. E. O'Brien, B. O'Gorman, E. Ostby, A. Petukhov, H. Putterman, C. Quintana, P. Roushan, N. C. Rubin, D. Sank, A. Skolik, V. Smelyanskiy, D. Strain, M. Streif, M. Szalay, A. Vainsencher, T. White, Z. J. Yao, P. Yeh, A. Zalcman, L. Zhou, H. Neven, D. Bacon, E. Lucero, E. Farhi, and R. Babbush, "Quantum approximate optimization of non-planar graph problems on a planar superconducting processor," Nat. Phys. **17**, 332–336 (2021).
[19]S. Khairy, R. Shaydulin, L. Cincio, Y. Alexeev, and P. Balaprakash, "Learning to optimize variational quantum circuits to solve combinatorial problems," in *Proceedings of the AAAI Conference on Artificial Intelligence* (AAAI Press, 2020), Vol. 34, pp. 2367–2375.
[20]G. Acampora, A. Chiatto, and A. Vitiello, "Genetic algorithms as classical optimizer for the quantum approximate optimization algorithm," Appl. Soft Comput. **142**, 110296 (2023).
[21]A. Rad, A. Seif, and N. M. Linke, "Surviving the barren plateau in variational quantum circuits with Bayesian learning initialization," arXiv:2203.02464 (2022).
[22]D. J. Egger, J. Marecek, and S. Woerner, "Warm-starting quantum optimization," Quantum **5**, 479 (2021).
[23]R. Tate, M. Farhadi, C. Herold, G. Mohler, and S. Gupta, "Bridging classical and quantum with SDP initialized warm-starts for QAOA," ACM Trans. Quantum Comput. **4**, 1–39 (2023).
[24]R. Tate, J. Moondra, B. Gard, G. Mohler, and S. Gupta, "Warm-started QAOA with custom mixers provably converges and computationally beats Goemans-Williamson's Max-Cut at low circuit depths," Quantum **7**, 1121 (2023).
[25]F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using QUBO models," arXiv:1811.11538 (2018).

[26]M. Hodson, B. Ruck, H. Ong, D. Garvin, and S. Dulman, "Portfolio rebalancing experiments using the quantum alternating operator ansatz," arXiv: 1911.05296v1 (2019).
[27]H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," Adv. Neural Inf. Process. Syst. **31**, 6391–6401 (2018).
[28]M. Stechły, L. Gao, B. Yogendran, E. Fontana, and M. Rudolph, "Connecting the Hamiltonian structure to the QAOA energy and Fourier landscape structure," arXiv:2305.13594v1 (2023).
[29]H. Jing, Y. Wang, and Y. Li, "Data-driven quantum approximate optimization algorithm for power systems," Commun. Eng. **2**, 12 (2023).
[30]C. Roch, D. Ratke, J. Nüßlein, T. Gabor, and S. Feld, "The effect of penalty factors of constrained Hamiltonians on the eigenspectrum in quantum annealing," ACM Trans. Quantum Comput. **4**, 1–18 (2023).
[31]J. S. Baker and S. K. Radha, "Wasserstein solution quality and the quantum approximate optimization algorithm: A portfolio optimization case study," arXiv:2202.06782 (2022).
[32]*Proceedings of the Genetic and Evolutionary Computation Conference Companion*, edited by J. E. Fieldsendand M. Wagner (ACM, New York, NY, 2022).
[33]J. A. Nelder and R. Mead, "A simplex method for function minimization," Comput. J. **7**, 308–313 (1965).
[34]F. Gao and L. Han, "Implementing the Nelder-Mead simplex algorithm with adaptive parameters," Comput. Optim. Appl. **51**, 259–277 (2012).
[35]M. J. D. Powell, "An efficient method for finding the minimum of a function of several variables without calculating derivatives," Comput. J. **7**, 155–162 (1964).
[36]M. R. Hestenes and E. Stiefel, "Methods of conjugate gradients for solving linear systems," J. Res. Natl. Bur. Stand. **49**, 409 (1952).
[37]C. G. Broyden, "The convergence of a class of double-rank minimization algorithms 1. General considerations," IMA J. Appl. Math. **6**, 76–90 (1970).
[38]R. S. Dembo and T. Steihaug, "Truncated-Newton algorithms for large-scale unconstrained optimization," Math. Program. **26**, 190–212 (1983).
[39]M. J. D. Powell, "A direct search optimization method that models the objective and constraint functions by linear interpolation," in *Advances in Optimization and Numerical Analysis*, edited by S. Gomez and J.-P. Hennart (Springer, Dordrecht, The Netherlands, 1994), Vol. 7, pp. 51–67.
[40]J. C. Spall, "An overview of the simultaneous perturbation method for efficient optimization," John Hoplins APL Technical Digest, 1998.
[41]D. J. Wales and J. P. K. Doye, "Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms," J. Phys. Chem. A **101**, 5111–5116 (1997).
[42]R. Storn and K. Price, "Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces," J. Global Optim. **11**, 341–359 (1997).
[43]Y. Xiang, D. Sun, W. Fan, and X. Gong, "Generalized simulated annealing algorithm and its application to the Thomson model," Phys. Lett. A **233**, 216–220 (1997).
[44]S. C. Endres, C. Sandrock, and W. W. Focke, "A simplicial homology algorithm for Lipschitz optimisation," J. Global Optim. **72**, 181–217 (2018).